portada.pdf

Teoría de la Computación y Lenguajes Formales: Sesión 13. Anatomía del Motor de Redes de Petri: Evolución frente a los Motores DFA y PDA

Alcance de la sesión 13

La Evolución de los Tres Motores de Ejecución

Anatomía Interna del Motor de Redes de Petri

fondo1280x720.pdf

Comparativa de Implementación en Código Python

Capacidades Avanzadas del Motor en la Industria 4.0

Conclusión

fondo1280x720.pdf

La Evolución de los Tres Motores: De la Máquina de Estados a la Red Concurrente

Los Tres Niveles de Motores en el Curso

De la Jerarquía Formal a la Arquitectura de Software

A lo largo del semestre hemos construido tres motores ejecutables en Python, cada uno respondiendo a un nivel de complejidad computacional y de modelado de procesos:

1. Motor DFA (`recurso.py`)

- **Base Formal:** Autómata Finito Determinista (Lenguajes Regulares).
- **Estado:** Escalar único $q \in Q$.
- **Alcance:** Procesos locales estrictamente secuenciales.

2. Motor PDA (`recurso_pda.py`)

- **Base Formal:** Autómata con Pila (Lenguajes Libres de Contexto).
- **Estado:** Tupla $(q, \text{stack}) \in Q \times \Gamma^*$.
- **Alcance:** Sub-procesos recursivos y jerarquías anidadas.

3. Motor RdP (`recurso_petri.py`)

- **Base Formal:** Redes de Petri y Álgebra Lineal Discreta.
- **Estado:** Vector de marcado $M \in \mathbb{N}^{|P|}$.
- **Alcance:** Concurrencia real, sincronización y recursos compartidos.

Matriz Comparativa de Paradigmas de Ejecución

Diferencias Estructurales de los Motores

Criterio	Motor DFA	Motor PDA	Motor RdP (Fénix)
Tipo de Estado	Escalar discreto q	Par $(q, \text{pila LIFO})$	Vector de marcado $M \in \mathbb{N}^{\lvert P \rvert}$
Memoria	Nula (sin historia)	Pila (acceso al tope Z)	Espacial / Distribuida en la red
Transición	Mapeo directo: $q' = \delta(q, e)$	Regla PUSH/POP: $(q', \alpha) = \delta(q, e, Z)$	Ecuación matricial: $M_{k+1} = M_k + C \cdot u_k$
Concurrencia	Imposible (monohilo)	Suspensión de contexto (pseudoconcurrencia)	**Nativa:** Múltiples actividades independientes activas
Recursos	Estados implícitos	Nivel en la pila	**Explícitos:** Lugares con fichas e invariantes de conservación

¿Por qué el Motor PDA no fue suficiente para Concurrencia?

La Limitación Intrínseca de la Pila (LIFO)

El autómata con pila resolvió la recursividad y el balance sintáctico de los sub-procesos (Clase 10), pero:

- **Flujo Estrictamente Secuencial:** Para entrar a una sub-tarea B, el contexto A se suspende en la pila; al terminar B, se desapila para retomar A.
- Imposibilidad de Ramificación Paralela (AND-Split): Una pila LIFO no puede dividirse en dos pilas autónomas que operen al mismo tiempo y luego se sincronicen (AND-Join).
- Contención de Recursos Compartidos: Si dos tareas independientes necesitan el mismo Torno CNC, una pila única no puede modelar la exclusión mutua sin serializar artificialmente el sistema.

El Salto Cualitativo hacia la Red de Petri

El estado deja de ser ≪*en qué paso estoy*≫ y pasa a ser ≪**dónde se encuentran todas las entidades, piezas y recursos en la fábrica**≫.

Anatomía Interna del Motor de Redes de Petri:

fondo1280x720.pdf

Matrices, Ecuación de Estado y Despacho

Estructura de Datos del Motor de RdP

Carga de la Especificación Formal (YAML / JSON)

A diferencia del DFA/PDA que leen tablas de adyacencia, el motor de RdP compila el archivo en una **representación algebraica matricial**:

1. Matrices de Incidencia Pre y Post

f

- **Matriz de Pre-condición (I^-):** Dimensiones $|P| \times |T|$. Define qué fichas consume cada transición:

$$I^-(p,t) = W(p,t)$$

- **Matriz de Post-condición (I^+):** Define qué fichas produce cada transición:

$$I^+(p,t) = W(t,p)$$

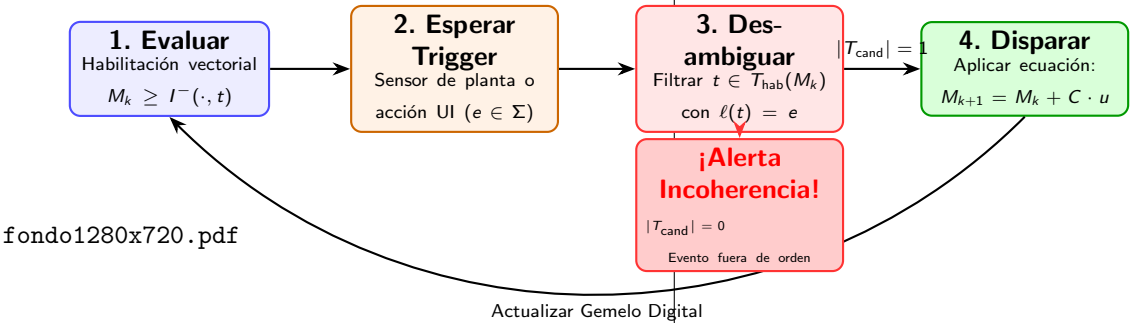
2. Matriz de Incidencia Compuesta (C)

Representa la variación neta de marcado:

$$C = I^+ - I^-$$

- Filas = Lugares (Procesos y Recursos).
- Columnas = Transiciones (Eventos/Tareas).
- Valores: $+1$ (crea ficha), -1 (consume ficha), 0 (neutro).

El Ciclo de Ejecución del Motor (*Execution Loop*)



fondo1280x720.pdf

La Diferencia Fundamental con los Motores Previos

El motor de RdP no cambia de estado siguiendo un puntero rígido; **evalúa continuamente qué transiciones son matemáticamente viables en paralelo**, permitiendo que ocurran eventos en cualquier orden legítimo.

Despacho de Triggers y Determinismo por Marcado

El Algoritmo de Despacho en Fénix

Cuando el bus de eventos de la fábrica emite una señal:

$$\text{Evento Recibido: } \langle \text{trigger} = \texttt{'FIN_MECANIZADO'}, \text{timestamp} = 10{:}15{:}00 \rangle$$

Cálculo de Transiciones Candidatas

El motor ejecuta la intersección lógica:

$$T_{\text{cand}} = \{t \in T \mid (M_k \geq I^-(\cdot, t)) \wedge (\ell(t) = \text{trigger})\}$$

- Si $|T_{\text{cand}}| = 1$: Se determina la transición única t^*. Se procede al disparo.
- Si $|T_{\text{cand}}| = 0$: Evento rechazado (sensor defectuoso o desincronización).

Ventaja Operacional

Permite que múltiples máquinas compartan nombres genéricos de eventos (ej: `'BOTON_OK'`), porque **el vector de marcado M_k resuelve unívocamente cuál máquina lo emitió**.

Comparativa de Implementación en Código: DFA vs. PDA vs. Redes de Petri

fondo1280x720.pdf

1. La Función de Transición en el Motor DFA (recurso.py)

Lógica Simple: Consulta a Diccionario de Estados

El estado es un string escalar; la transición es un salto directo:

```
class DFA:
    def transition(self, event):
        # 1. Verificar si el evento pertenece al alfabeto
        if event not in self.alphabet:
            raise ValueError(f"Evento ’{event}’ invalido.")

        # 2. Consultar directamente el diccionario: delta(estado, evento)
        state_transitions = self.transitions.get(self.current_state, {})
        if event in state_transitions:
            self.current_state = state_transitions[event] # Salto escalar
        else:
            raise ValueError("Transicion no permitida en este estado.")
```

fondo1280x720.pdf

Limitación: Solo un estado activo. No hay soporte para tareas simultáneas ni memoria previa.

2. La Función de Transición en el Motor PDA (recurso_pda.py)

Lógica con Contexto: Evaluación del Tope de Pila (LIFO)

El estado requiere validar la tupla (estado, evento, tope_pila):

```
class PDA:
    def transition(self, event):
        stack_top = self.stack[-1] # Leer memoria LIFO
        key = (self.current_state, event, stack_top)
fondo1280x720.pdf
        if key not in self.transitions:
            raise ValueError("Violacion de contexto: operacion fuera de subproceso.")

        new_state, push_symbols = self.transitions[key]
        self.stack.pop() # POP del simbolo actual
        for sym in reversed(push_symbols):
            self.stack.append(sym) # PUSH de nuevos contextos

        self.current_state = new_state
```

Mejora: Soporta jerarquías anidadas. Limitación: Sigue siendo monohilo y secuencial.

3. El Ciclo de Disparo en el Motor RdP (recurso_petri.py)

Lógica Vectorial y Matricial: Evaluación de Precondiciones y Ecuación de Estado

```
class PetriNet:
    def is_enabled(self, tid):
        t = self.t_idx[tid]
        # Validar condicion vectorial: M >= I-(:, t)
        return all(self.M[p] >= self.I_minus[p][t] for p in range(self.num_p))

    def disparar_por_trigger(self, trigger_name):
        # Desambiguar: transiciones habilitadas que coinciden con el trigger
        candidates = [t for t in self.get_enabled_transitions()
                      if self.transitions[t]['trigger'] == trigger_name]
        if len(candidates) != 1:
            raise RuntimeError("Incoherencia fisico-digital o ambiguedad.")

        # Ecuacion de estado algebraica: M_{k+1} = M_k + C * u_k
        t_id = candidates[0]
        for p in range(self.num_p):
            self.M[p] += self.C[p][self.t_idx[t_id]]
```

fondo1280x720.pdf

Potencia: Múltiples piezas, recursos y tareas coexisten en el mismo vector M.

Capacidades Avanzadas del Motor en la Industria 4.0: Supervisión, Watchdogs e Invariantes

fondo1280x720.pdf

Supervisión de Invariantes de Recursos en Tiempo Real

Verificación Continua de Leyes Físicas

El motor de Redes de Petri puede auditar en cada paso la invariabilidad estricta de sus recursos:

$$y^T \cdot M_k = y^T \cdot M_0 = \text{Constante}$$

Ejemplo: Conservación de un Robot

fondo1280x720.pdf

En una celda de manufactura:

$$M(\texttt{Robot_Libre}) + M(\texttt{Robot_Ocupado}) = 1$$

- Si la suma da 0: El recurso se perdió (bloqueo / fuga de tokens).
- Si la suma da > 1: El recurso se duplicó ilegalmente.

Autodiagnóstico por Álgebra

Como $y^T \cdot C = 0$, el motor garantiza que ninguna transición legal puede romper el invariante. Cualquier desvío revela una **falla física o manipulación no autorizada**.

Temporización y Centinelas (*Watchdogs*) en Fénix

Supervisión Proactiva vs. Auditoría Reactiva

En los motores DFA y PDA clásicos, el tiempo no formaba parte del modelo de estados.

Temporización Integrada (T-timed)

- Cada transición porta una duración nominal $\tau(t)$.
- El motor calcula el tiempo acumulado de valor añadido vs. tiempo de espera en cola por falta de recurso.

Temporizador Centinela (*Watchdog*)

- Al entrar una ficha en un lugar, se activa una alarma si:

$$t_{\text{actual}} - t_{\text{entrada}} > \tau_{\text{nom}} + \Delta_{\text{tol}}$$

- La alerta se genera **mientras la demora está sucediendo en planta**.

Conclusión de la Sesión 13

Lo que aprendimos en esta sesión

- **La Evolución del Estado:** De un escalar q (DFA) a una tupla con memoria LIFO (q, pila) (PDA), y finalmente a un vector multidimensional distribuido $M \in \mathbb{N}^{|P|}$ (RdP).
- **El Motor Basado en Álgebra Lineal:** La función de transición es sustituida por la verificación vectorial ($M \geq I^-$) y la ecuación de estado ($M_{k+1} = M_k + C \cdot u$).
- **Determinismo Condicionado al Marcado:** El despacho de triggers permite a operadores y sensores usar nombres genéricos de eventos sin ambigüedad en ejecución.
- **El Gemelo Digital en Acción:** El motor de RdP (Fénix) soporta nativamente la planificación previa, supervisión en tiempo real con watchdogs y diagnóstico causal de demoras.

Próximo Bloque (Semana 14): Control Supervisor Descentralizado (SCT)

Teoría de Ramadge-Wonham: Eventos controlables vs. incontrolables (Σ_c, Σ_{uc}), composición síncrona Planta $\|$ Supervisor y prevención formal de deadlocks en arquitecturas distribuidas.

¡Muchas gracias!

salida.pdf